

■ 2.4.6 Threading Expressions

When you have a structure like a list, it is often convenient to apply a particular function “in parallel” to all the elements of the list. Functions that have the attribute `Listable` automatically get applied in this way. You can use `Thread` to do this with any function.

`Thread[f[{a1, a2}, {b1, b2}]]` thread *f* over lists to give {*f*[a₁, b₁], *f*[a₂, b₂]}
`Thread[f[args], g]` thread *f* over objects with head *g* in *args*

Functions for threading expressions.

Here is a function whose arguments are lists.

```
In[1]:= f[{a1, a2}, {b1, b2}]
Out[1]= f[{a1, a2}, {b1, b2}]
```

`Thread` applies the function “in parallel” to each element of the lists.

```
In[2]:= Thread[%]
Out[2]= {f[a1, b1], f[a2, b2]}
```

Arguments that are not lists get copied.

```
In[3]:= Thread[ f[{a1, a2}, {b1, b2}, c, d] ]
Out[3]= {f[a1, b1, c, d], f[a2, b2, c, d]}
```

Most built-in mathematical functions have the attribute `Listable`, and so automatically get threaded over lists.

```
In[4]:= Log[{a, b, c}]
Out[4]= {Log[a], Log[b], Log[c]}
```

`Log` does not automatically get applied to both sides of structures other than lists, such as equations.

```
In[5]:= Log[x == y]
Out[5]= Log[x == y]
```

You can use `Thread` to apply functions to both sides of an equation.

```
In[6]:= Thread[%, Equal]
Out[6]= Log[x] == Log[y]
```